

Week 09 Explained Student Worksheet

Evaluation of Information Retrieval Systems

Name _____ Student ID _____ Group _____

Evaluate the Week 8 ranking against explicit relevance judgments. Compare a scored ranking and a filename-order baseline using the same queries and cutoffs.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Identify true positives, false positives, and false negatives.
- Implement precision, recall, F1, Precision@k, and Recall@k.
- Evaluate computed rankings against predefined relevance sets.
- Compare two outputs fairly and state the limits of the comparison.

Understand the concepts

Retrieved documents are the IDs returned under the chosen retrieval policy. Relevant documents are IDs judged useful for a particular information need. Relevance is not inferred from a score. The JSON file defines three queries, their intents, and their relevant document IDs for this teaching collection.

TP counts retrieved IDs in the relevance set. FP counts retrieved IDs outside it. FN counts relevant IDs not retrieved. Precision asks what fraction of retrieved documents is relevant; recall asks what fraction of all relevant documents was retrieved. F1 is their harmonic mean.

This lab defines a zero result for precision when nothing is retrieved, recall when the relevance set is empty, and F1 when precision plus recall is zero. These are reporting conventions for undefined ratios; an empty relevance set should be flagged when interpreting an evaluation.

Precision@k is the number of relevant unique IDs in the first k positions divided by k. If fewer than k results are returned, unfilled positions count as nonrelevant. Recall@k divides the same relevant count by the total number of relevant IDs. k must be positive. Duplicate IDs are removed while keeping their first appearance.

Both methods use $k = 3$. The TF-IDF method returns only positive-score documents from the Week 8 ranking; the baseline orders all documents by filename. Ordinary precision/recall/F1 in the report are calculated on the available top-three IDs. P@3 can differ from ordinary precision when fewer than three results exist.

Predict before running

If three IDs are retrieved and two are relevant out of four relevant documents, identify TP, FP, and FN.

Prepare your terminal

Extract the course and open a terminal inside Week_09_Retrieval_Evaluation. Keep docs and data beside the Python files. The main demonstration is lab09_solution.py; your exercise starter is lab09_exercises.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab09_solution.py
py -3.13 lab09_exercises.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

Use the same relevance set for both outputs: relevant = {d1, d3, d5}. Output A is [d1, d2, d3]; output B is [d1, d4, d2]. Calculate the metrics before executing the example.

```
relevant = {"d1", "d3", "d5"}
outputs = {"A": ["d1", "d2", "d3"],
          "B": ["d1", "d4", "d2"]}
for label, retrieved in outputs.items():
    tp = len(set(retrieved) & relevant)
    p = tp / len(retrieved)
    r = tp / len(relevant)
    f1 = 0 if p + r == 0 else 2 * p * r / (p + r)
    print(label, f"P={p:.4f} R={r:.4f} F1={f1:.4f}")
```

Expected output from an executed example

```
A P=0.6667 R=0.6667 F1=0.6667
B P=0.3333 R=0.3333 F1=0.3333
```

How each stage produces the result

1. The relevant set is specified before evaluating outputs. Output A contains two relevant IDs and one nonrelevant ID, so TP=2, FP=1, and FN=1.
2. Precision divides TP by the number retrieved. Recall divides TP by the number relevant. In this example both denominators are 3, but they need not be equal.
3. F1 combines precision and recall using their harmonic mean. The branch for p+r=0 avoids division by zero when neither metric is positive.
4. The comparison applies only to this judged example. It does not prove a method is better for every query or collection. Judge intent first, then evaluate the ranking.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab09_solution.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def precision(retrieved, relevant):
    retrieved = set(retrieved)
    relevant = set(relevant)
    if not retrieved:
        return 0.0
    return len(retrieved & relevant) / len(retrieved)

def recall(retrieved, relevant):
    relevant = set(relevant)
    if not relevant:
        return 0.0
    return len(set(retrieved) & relevant) / len(relevant)

def precision_at_k(ranked_docs, relevant_docs, k):
    if k <= 0:
        raise ValueError("k must be positive")
    top = unique_ranking(ranked_docs)[:k]
    # Unfilled positions count as nonrelevant: denominator stays k.
    return len(set(top) & set(relevant_docs)) / k
```

Code walkthrough

1. precision and recall convert IDs to sets so duplicate IDs cannot inflate counts. The course returns 0 for an empty retrieved list or an empty relevance set.
2. precision_at_k requires positive k. unique_ranking removes duplicate IDs without changing first-occurrence rank order.
3. P@k keeps k as the denominator even if fewer than k results are available. Ordinary precision divides by the number actually retrieved, so the two values can differ.
4. The lab filters ranked results to positive scores and compares them with a filename-order baseline using identical judgments and k. retrieval.py supplies the Week 8 ranking code.

With one relevant result returned and k=3, compare ordinary precision with P@3. State both denominators.

Connection to the course

The project can evaluate only queries with supplied relevance judgments. A new query can still be searched, but a metric requires a defined information need and relevant set.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Review each query intent and relevance set in data/relevance.json before looking at the scores.

Inspect the relevance JSON first. These are explicit teaching judgments, not labels inferred from similarity scores.

Step 2 Run the Week 8 retrieval implementation copied into retrieval.py; keep all ten documents unchanged.

Use the same ten documents and ranking implementation for both evaluation methods.

Step 3 Retain only positive-score IDs and take the first three. Independently count TP, FP, and FN.

List top IDs and mark each relevant or nonrelevant before calculating metrics.

Step 4 Implement the metric functions and compare your values with the program.

Use the documented empty-set and short-ranking conventions in your own functions.

Step 5 Evaluate the filename-order baseline using the same relevance sets and k.

Keep the query, collection, judgments, and cutoff fixed so the comparison has a clear interpretation.

Step 6 Explain any difference between precision and P@3 when the ranking has fewer than three positive results. Report findings per query and avoid universal claims.

Report a result per query. If fewer than three positive matches exist, explain the two precision denominators explicitly.

Test record

Check to demonstrate	Actual observation	Pass or revise
TP, FP, and FN match manually marked document IDs		
Duplicate IDs do not inflate the metrics		
A short ranking uses k as the P@k denominator		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Hand calculation: finish the A/B example and compute $P@2$ and $R@2$ for each output.
- Implementation: complete `precision_at_k` in the starter using the documented short-list convention.
- Debugging: explain why dividing recall by the number of retrieved documents gives the wrong measure.
- Interpretation: test an empty retrieved list, an empty relevant set, and k larger than the ranking.

Diagnose common errors

Recall equals precision for the wrong reason. Recall uses the number of relevant documents, not the number returned. Equal values are possible, but the denominators must be checked separately.

$P@3$ becomes 1 for one returned relevant ID. Under the stated convention, two unfilled positions count as nonrelevant. $P@3$ is $1/3$, while ordinary precision is 1.

Submit these items

- Completed metric functions and hand-checked TP/FP/FN counts for each query.
- A table comparing both methods under the same relevance set and cutoff.
- A short conclusion limited to this collection, these judgments, and the chosen cutoff.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

What distinguishes relevance from similarity?

What is a false negative?

Optional challenge

Compute macro-averaged precision, recall, and F1 over the three queries. Compare mean per-query F1 with F1 calculated from mean precision and mean recall; explain why they need not match.

Source for this companion

Based on the supplied Week 09 lab sheet, shipped program, exercise starter, and sample data. The complete main program and `expected_output.txt` in this folder are the references for checking your baseline run.